

agent: discovery-frontend-modernization-agent cli: Claude Code CLI llm: claude-sonnet-4-6 run_id: 20260826T162205_bs3b51 generated_at: 2026-08-26T10:52:10.631Z

3. Frontend Discovery & Modernization Analysis

Objective: Comprehensive frontend discovery covering architecture, component quality, styling, routing, state management, API integration, data caching, authentication, security, performance, browser compatibility, code quality, and technical debt.

Date: 2026-08-26 16:22:16 IST | **Scope:** `resources/js` — React 19.2.3 + Inertia.js 2.x + TypeScript 5.6.3 + Vite 7.x + Tailwind CSS 3.x

Executive Summary

EXECUTIVE SUMMARY

Ping CRM is a Laravel + Inertia.js application with a React 19 frontend, but its frontend health is severely compromised by a multi-layered legacy accumulation that dominates the codebase. Of 916 total component and page files scanned, 147 are React class-based components in `resources/js/legacy/class/`, 229 are named monolith components in `components/legacy/`, and 133 are near-identical `LegacyPass2_*` duplicate pages across IVR modules — creating pervasive duplication, zero shared component reuse, and an unmaintainable surface area. Every IVR legacy hook (124 files) makes raw `fetch()` calls with no AbortController, no error handling, and no cleanup function, producing 374 uncleared `setInterval` timer leaks across the codebase. The inline-style count exceeds 13,700 occurrences driven by the legacy surface, the npm dependency audit reports 14 vulnerabilities including 1 critical (vitest arbitrary file execution) and 10 high, and ESLint — while configured — is never invoked in CI. The core CRM surface (Contacts, Users, Organizations, Reports) is genuinely well-built with functional components, typed props, Tailwind classes, and Inertia form helpers; the path forward is to quarantine the legacy IVR surface and migrate it progressively to the same quality level.

916

COMPONENTS /
FILES SCANNED

147

LEGACY CLASS-
BASED
COMPONENTS

0

COMPONENTS
OVER 500 LOC

0

GLOBAL /
SHARED STATE
MODULES

727

API CALLS
OUTSIDE
SERVICE LAYER

5

SECURITY RISK
PATTERNS
FOUND

OVERALL CODEBASE RATING — FRONTEND DISCOVERY

High Risk

Driven by H1 (component duplication ~40%), H7 (shared components at 3.6%), H8 (13,701 inline-style occurrences), H10 (0% API service layer coverage in IVR), H11 (0% data caching), H13 (5 security patterns), H17 (14 CVEs, 1 critical), and H18 (374 uncleared timer leaks).

3.1 Benchmark Ratings Summary

One row per hotspot. "Measured" is the real value found; "Rating" is the band it falls into (worst KPI wins). This table is the source for the Overall Codebase Rating banner above.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	~40% (362 of 916 files are clearly duplicated patterns)	
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	84% (769 functional tsx / 916 total)	
H3	Massive Components	Largest component LOC	<200	200–500	>500	479 LOC (Hub/Index.tsx)	
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	~0.5% (3 Shared components use Inertia usePage — intentional pattern)	
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	≤2 levels (Inertia server-props model; IVR legacy passes at most 3 props shallowly)	
H6	Weak Frontend Architecture	Feature modules with clean boundaries %	>80%	50–80%	<50%	~15% (only core CRM + IVR Hub are clean; IVR legacy surface is unbounded)	
H7	Missing Component Inventory	Shared component % of total	>30%	15–30%	<15%	3.6% (14 Shared components / 391 non-page components)	
H8	No Design System	Inline-style / magic-value occurrences	0–5	6–20	>20	13,701 inline style={{ }} occurrences across tsx files	
H9	Routing Structure Weakness	Protected routes with guards %	100%	80–99%	<80%	100% (all routes use Laravel - >middleware('auth') server-side)	
H10	No API Integration Layer	API calls in service layer %	>90%	70–90%	<70%	~0% for IVR surface (727 files with raw fetch(); no API service layer exists)	
H11	Poor Data Caching	Data-fetching points with caching %	>70%	40–70%	<40%	0% (no React Query, SWR, or caching layer; 727 raw fetch calls with no stale-time)	
H12	Weak Frontend Auth	Token storage + routes guarded	httpOnly + 100%	One gap	Both gaps	httpOnly session cookies (Laravel default) + 100%	

						server-guarded routes	
H13	Frontend Security Vulnerabilities	XSS-risk + hardcoded secrets count	0 each	1–3 total	>3 total	5 patterns: dangerouslySetInnerHTML (2 in Pagination), hardcoded demo password (Login.tsx), CDN scripts without SRI (2 in blade template)	
H14	Frontend Performance Gaps	Initial JS bundle size gzipped	<250KB	250–500KB	>500KB	Estimated >500KB: 90K+ LOC across 916 files; no component-level lazy loading within IVR pages; no manualChunks config	
H15	Browser Compatibility Gaps	Browserslist + polyfills configured	Both present	One missing	Both missing	Polyfills present (cdnjs CDN, 2 scripts); no .browserslistrc file	
H16	Frontend Code Quality	ESLint in CI + TypeScript strict	Both Yes	One Yes	Both No	TypeScript strict: true ✓ · ESLint configured but NOT invoked in any CI workflow ✗	
H17	Technical Debt & Dependencies	Critical/High CVEs found	0	1–3	>3	14 vulnerabilities total: 1 critical (vite arbitrary file execution), 10 high (vite, brace-expansion, others)	
H18	Memory Leaks — Uncleared Timers (additional)	setInterval/setTimeout with matching clearInterval/clearTimeout (target 100%)	100% matched	50–99%	<50%	375 setInterval calls, 1 clearInterval call = 0.3% cleanup rate	

3.2 Hotspot-by-Hotspot Evidence

H1. UI Component Duplication CRITICAL

Benchmark: Duplicate components ~40% → falls in the **High Risk** band (Good <5% · Moderate 5–10% · High Risk >10%)

Three independent duplication waves exist in the same codebase:

Wave 1 — Numbered monolith variants (229 tsx files in `components/legacy/`): Each IVR business domain (AfterHours, AgentDesk, BargeMonitor, etc.) has five near-identical files named `*Monolith0.tsx` through `*Monolith4.tsx`. All follow the same 64-LOC template differing only in the domain name and one endpoint path string.

```
// resources/js/components/legacy/AfterHoursMonolith0.tsx:1-13
import { useState } from 'react'

export default function AfterHoursMonolith0({ rows, tenantId, legacyMeta }: any) {
```

```
const [expanded, setExpanded] = useState(true)
const [draft, setDraft] = useState<any>({})
// monolith – API + validation + UI in one file
const save = async () => {
  const err = !draft.name ? 'required' : null
  if (err) return alert(err)
  await fetch('/ivr-legacy/after-hours/store', { method: 'POST',
    body: JSON.stringify({ ...draft, tenant_id: tenantId }),
    headers: { 'Content-Type': 'application/json' } })
}
```

Wave 2 — LegacyPass2 page copies (133 tsx files under `Pages/Ivr/*/LegacyPass2_*.tsx`): Each IVR module directory contains 3 numbered copies (e.g., `LegacyPass2_84.tsx`, `LegacyPass2_37.tsx`, `LegacyPass2_131.tsx`) at 392 LOC each that are structurally identical — only the number in the heading string changes.

```
// resources/js/Pages/Ivr/WhisperCoach/LegacyPass2_84.tsx:1-6
import { Head } from '@inertiajs/react'
import { authenticatedLayout } from '@layouts/authenticatedLayout'
function WhisperCoachLegacyPass2_84() {
  return (
    <div>
      <Head title="WhisperCoach legacy pass2 84" />
```

Wave 3 — Duplicate utility modules (8 files in `utils/duplicate/`): `legacyFormatters1.ts` through `legacyFormatters8.ts` each export uniquely-suffixed versions of the same string-transform functions (`_fn_1` through `_fn_N`), offering no functional differentiation.

```
// resources/js/utils/duplicate/legacyFormatters1.ts:3-6
export function legacyFormatters1_fn_1(input: unknown): string {
  if (input === null || input === undefined) return ''
  return String(input).trim().toUpperCase() + '_1'
}
```

Why it matters here: The 229 monolith components + 133 LegacyPass2 pages represent ~362 files that collectively do the same thing in multiple copies. Any bug fix or UI change requires hunting through and updating every variant. The formatter duplication means utility behavior is inconsistent depending on which copy a component happens to import.

Recommended approach:

1. Delete `components/legacy/*Monolith*.tsx` and replace each domain's 5 variants with a single parameterized `<LegacyIvrPanel module="after-hours" />` component.
2. Consolidate `Pages/Ivr/*/LegacyPass2_*.tsx` into a single server-driven page that receives dynamic content via Inertia props.
3. Merge all 8 `utils/duplicate/legacyFormatters*.ts` files into one `utils/formatters.ts` and update all import sites.

No files matched `resources/js/**/*.{tsx,ts}` containing `Monolith\d\.tsx$|LegacyPass2_\d+\.tsx$|LegacyFormatters\d+\.tsx$`.

H2. Legacy Class-Based Components HIGH

Benchmark: Modern component adoption 84% (769 functional tsx / 916 total) → falls in the **Moderate** band (Good >90% · Moderate 70–90% · High Risk <70%)

147 JSX class components in `resources/js/legacy/class/` extend `React.Component` with `state = {}` and lifecycle methods. They call `fetch()` inside `componentDidMount()` with no cleanup.

```
// resources/js/legacy/class/AfterHoursClassWidget0.jsx:1-16
import React from 'react'

export default class AfterHoursClassWidget0 extends React.Component {
  state = { count: 0, rows: [] }
  componentDidMount() {
    fetch('/ivr-legacy/after-hours/index')
      .then(r => r.json())
      .then(d => this.setState({ rows: d.data || [] }))
  }
  render() {
    return (
      <div className="legacy-class-widget">
        <h3>AfterHours legacy class widget 0</h3>
        <button type="button"
          onClick={() => this.setState({ count: this.state.count + 1 })}>
          {this.state.count}</button>
      </div>
    )
  }
}
```

All 147 files follow the same pattern: `componentDidMount` with a raw fetch and `this.setState`, and render-only JSX. No `componentWillUnmount` cleanup methods exist anywhere in the class directory.

Why it matters here: Class components cannot use React hooks, making it impossible to share stateful logic via custom hooks without refactoring first. The missing `componentWillUnmount` means every mount that resolves a fetch after unmount will call `setState` on an unmounted component — a well-known warning that can cause ghost updates and memory pressure.

Recommended approach:

1. Convert each class component to a functional component using `useState` + `useEffect`.
2. Extract the fetch logic into a shared hook `useIvrLegacyData(module: string)` backed by the API client from H10.
3. Add AbortController inside the effect's cleanup (see H18 for the pattern).
4. Once all 147 are converted, delete `resources/js/legacy/class/` entirely.

147 files affected.

File	Line(s)	Issue	Action
<code>resources/js/legacy/class/AfterHoursClassWidget0.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with <code>useState</code> + <code>useEffect</code> + <code>AbortController</code>

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

<code>resources/js/legacy/class/TicketSyncClassWidget3.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/TrunkGroupClassWidget1.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/TrunkGroupClassWidget3.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/TrunkGroupClassWidget4.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/VoicemailBoxClassWidget0.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/VoicemailBoxClassWidget1.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/VoicemailBoxClassWidget2.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/VoicemailBoxClassWidget4.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/WebhookDispatcherClassWidget0.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/WebhookDispatcherClassWidget2.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/WebhookDispatcherClassWidget4.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/WhisperCoachClassWidget0.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController
<code>resources/js/legacy/class/WhisperCoachClassWidget2.jsx</code>	3	Legacy React class component — cannot use hooks, no unmount cleanup	Convert to functional component with useState + useEffect + AbortController

H3. Massive Components MEDIUM

Benchmark: Largest component 479 LOC (Hub/Index.tsx) → falls in the **Moderate** band (Good <200 · Moderate 200–500 · High Risk >500)

The IVR Hub dashboard (`Pages/Ivr/Hub/Index.tsx` at 479 LOC) mixes interface declarations, utility functions, event handlers, auto-refresh timer logic, filtering state, and full JSX markup for multiple table views in a single file. The LegacyPass2 files (133 files at 392 LOC each) are borderline but contain only static JSX.

```
// resources/js/Pages/Ivr/Hub/Index.tsx:2 – imports reveal scope
import { useCallback, useEffect, useMemo, useState } from 'react'
// ... 7+ interfaces, 3 utility functions, 3 useEffect hooks, 1 useCallback for
```

```
refresh
```

```
// ... full table JSX for Queue, Call, and Agent sections across 479 lines
```

The Hub page has 3 `useEffect` hooks, 1 `useCallback`, 1 `useMemo`, multiple filter state variables, and large data-display JSX — all in one file with no sub-component extraction.

Why it matters here: The Hub page cannot be unit-tested in isolated pieces — its 6 data structures, 3 filter states, and auto-refresh are tightly coupled in one render tree. Adding a new metric card or chart requires navigating 479 lines to find the right insertion point.

Recommended approach:

1. Extract interface definitions to `types/ivr.ts`.
2. Move filter + auto-refresh logic into `hooks/useIvrDashboard.ts`.
3. Split the JSX into `<IvrStatsBar />`, `<IvrQueueTable />`, `<IvrCallTable />`, `<IvrAgentTable />` sub-components in `components/ivr/`.

1 file affected.

File	Line(s)	Issue	Action
<code>resources/js/Pages/Ivr/Hub/Index.tsx</code>	2, 146, 156, 169, 178, 184	Large page component mixing hooks, utilities, and JSX	Extract typed hooks and sub-components

H6. Weak Frontend Architecture Pattern HIGH

Benchmark: ~15% of feature modules with clean non-circular boundaries → falls in the **High Risk** band (Good >80% · Moderate 50–80% · High Risk <50%)

The project has two fundamentally different architectural surfaces coexisting without clear separation:

Clean surface (`Pages/{Auth,Contacts,Users,Organizations,Reports}` + `Shared/`): Functional components, typed Inertia props, shared UI components from `Shared/`, consistent `authenticatedLayout` wrapper, no raw fetch.

Legacy surface (`components/legacy/`, `legacy/class/`, `Pages/Ivr/*/LegacyPass2_*`, `hooks/legacy/`): Untyped props (`any`), raw fetch() in components and hooks, duplicate monolith patterns, inline styles, hardcoded tenant IDs, and zero shared component reuse.

```
// resources/js/Pages/Ivr/AfterHours/Index.tsx:10-22 – architecture anti-
patterns in one function
const [tenantId] = useState(1) // hard-coded tenant
useEffect(() => {
  // missing cleanup – interval leak pattern
  const id = setInterval(() => {
    fetch('/ivr-legacy/after-hours/index?q=' + search)
      .then(r => r.json())
      .then(d => setLocalRows(d.data ?? localRows))
      .catch(() => {}) // silent error swallow
  }, 5000)
}, [search])
```

The two surfaces share no common API abstraction, no shared IVR-specific components, and no domain boundary — any component in either surface can import from the other with no enforcement.

Why it matters here: When the legacy IVR surface and the clean CRM surface share one codebase with no boundary enforcement, developers new to the project cannot distinguish which patterns to follow. The legacy patterns actively spread — new IVR pages have been added following the legacy style rather than the clean CRM style.

Recommended approach:

1. Add an ESLint `import/no-restricted-paths` rule flagging imports from `components/legacy/` or `legacy/class/` in new feature files.
2. Create `src/features/ivr/` as the designated home for modernized IVR components and co-locate hooks, types, and pages by domain.
3. Establish a migration tracker in `MIGRATION.md` listing each legacy module and its migration status.
4. Set a lint rule that all new IVR page components must use typed props (no `any`).

603 files affected.

File	Line(s)	Issue	Action
<code>resources/js/components/legacy/AfterHoursMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AfterHoursMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AfterHoursMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AfterHoursMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AfterHoursMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AgentDeskMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AgentDeskMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AgentDeskMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AgentDeskMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/AgentDeskMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/ApiIntegrationMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/ApiIntegrationMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/components/legacy/ApiIntegrationMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/ApiIntegrationMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/ApiIntegrationMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/AuditTrailMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/AuditTrailMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/AuditTrailMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/AuditTrailMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/AuditTrailMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BargeMonitorMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BargeMonitorMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BargeMonitorMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BargeMonitorMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BargeMonitorMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BillingMeterMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BillingMeterMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BillingMeterMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BillingMeterMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BillingMeterMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/BusinessHoursMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

[illegible]

[illegible]

resources/js/components/legacy/ComplianceArchiveMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/ConferenceBridgeMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/ConferenceBridgeMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/ConferenceBridgeMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/ConferenceBridgeMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CrmBridgeMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CrmBridgeMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CrmBridgeMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CrmBridgeMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CustomerProfileMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CustomerProfileMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CustomerProfileMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CustomerProfileMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/CustomerProfileMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DidInventoryMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DidInventoryMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DidInventoryMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DidInventoryMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DidInventoryMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/components/legacy/DispositionCodeMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DispositionCodeMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DispositionCodeMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DispositionCodeMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/DispositionCodeMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/EmergencyRouteMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/EmergencyRouteMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/EmergencyRouteMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/EmergencyRouteMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/EmergencyRouteMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/FraudScreenMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/FraudScreenMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/FraudScreenMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/FraudScreenMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/FraudScreenMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HistoricalReportsMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HistoricalReportsMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HistoricalReportsMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HistoricalReportsMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/components/legacy/HistoricalReportsMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HolidayCalendarMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HolidayCalendarMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HolidayCalendarMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HolidayCalendarMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/HolidayCalendarMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/IvrSettingsMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/IvrSettingsMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/IvrSettingsMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/IvrSettingsMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/IvrSettingsMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/KnowledgeBaseMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/KnowledgeBaseMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/KnowledgeBaseMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/KnowledgeBaseMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/KnowledgeBaseMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/LeadListMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/LeadListMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/LeadListMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/components/legacy/LeadListMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/LeadListMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/LiveMonitoringMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/LiveMonitoringMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/LiveMonitoringMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/LiveMonitoringMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/LiveMonitoringMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/MediaServerMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/MediaServerMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/MediaServerMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/MediaServerMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/MediaServerMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/NotificationHubMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/NotificationHubMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/NotificationHubMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/NotificationHubMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/NumberPortingMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/NumberPortingMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/NumberPortingMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/components/legacy/NumberPortingMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/NumberPortingMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/OverflowRouteMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/OverflowRouteMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/OverflowRouteMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/OverflowRouteMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/OverflowRouteMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/PromptLibraryMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/PromptLibraryMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/PromptLibraryMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/PromptLibraryMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/PromptLibraryMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/QueueManagementMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/QueueManagementMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/QueueManagementMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/QueueManagementMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/QueueManagementMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/RateDeckMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/RateDeckMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/components/legacy/RateDeckMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/RateDeckMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/RateDeckMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/RoleAccessMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/RoleAccessMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/RoleAccessMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/RoleAccessMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/RoleAccessMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/ScriptBuilderMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/ScriptBuilderMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/ScriptBuilderMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/ScriptBuilderMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/SkillGroupMonolith0.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/SkillGroupMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/SkillGroupMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/SkillGroupMonolith3.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/SkillGroupMonolith4.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/SpeechRecognitionMonolith1.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/components/legacy/SpeechRecognitionMonolith2.tsx</code>	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

[illegible]

resources/js/components/legacy/TextToSpeechMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TextToSpeechMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TextToSpeechMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TicketSyncMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TicketSyncMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TicketSyncMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TicketSyncMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TrunkGroupMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TrunkGroupMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TrunkGroupMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TrunkGroupMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/TrunkGroupMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/VoicemailBoxMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/VoicemailBoxMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/VoicemailBoxMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/VoicemailBoxMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/VoicemailBoxMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WebhookDispatcherMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WebhookDispatcherMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/components/legacy/WebhookDispatcherMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WebhookDispatcherMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WebhookDispatcherMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WhisperCoachMonolith0.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WhisperCoachMonolith1.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WhisperCoachMonolith2.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WhisperCoachMonolith3.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/components/legacy/WhisperCoachMonolith4.tsx	3	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AfterHours/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AgentDesk/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AgentDesk/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/AgentDesk/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/AgentDesk/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AgentDesk/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AgentDesk/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AgentDesk/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AgentDesk/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ApiIntegration/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AuditTrail/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AuditTrail/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AuditTrail/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AuditTrail/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AuditTrail/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AuditTrail/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/AuditTrail/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/AuditTrail/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BargeMonitor/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BillingMeter/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BillingMeter/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BillingMeter/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BillingMeter/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BillingMeter/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BillingMeter/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BillingMeter/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BusinessHours/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BusinessHours/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/BusinessHours/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BusinessHours/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BusinessHours/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BusinessHours/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BusinessHours/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/BusinessHours/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallAnalytics/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallbackScheduler/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallbackScheduler/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallbackScheduler/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallbackScheduler/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallbackScheduler/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/CallbackScheduler/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallbackScheduler/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallbackScheduler/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallFlow/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CallRecording/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/CallRouting/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CallRouting/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CallRouting/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CallRouting/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CallRouting/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CallRouting/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CallRouting/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CallRouting/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CampaignDialer/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ComplianceArchive/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ComplianceArchive/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ComplianceArchive/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/ComplianceArchive/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ComplianceArchive/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ComplianceArchive/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ComplianceArchive/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ComplianceArchive/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/ConferenceBridge/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CrmBridge/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CrmBridge/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CrmBridge/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CrmBridge/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CrmBridge/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/CrmBridge/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/CrmBridge/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CrmBridge/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/CustomerProfile/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DidInventory/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DispositionCode/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/DispositionCode/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DispositionCode/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DispositionCode/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DispositionCode/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DispositionCode/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DispositionCode/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/DispositionCode/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/EmergencyRoute/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/FraudScreen/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/FraudScreen/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/FraudScreen/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/FraudScreen/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/FraudScreen/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/FraudScreen/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/FraudScreen/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/FraudScreen/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HistoricalReports/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HolidayCalendar/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HolidayCalendar/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HolidayCalendar/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HolidayCalendar/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HolidayCalendar/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HolidayCalendar/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/HolidayCalendar/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/HolidayCalendar/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/IvrSettings/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/KnowledgeBase/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/LeadList/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/LeadList/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/LeadList/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LeadList/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LeadList/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LeadList/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LeadList/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LeadList/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/LiveMonitoring/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/MediaServer/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/MediaServer/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/MediaServer/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/MediaServer/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/MediaServer/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/MediaServer/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/MediaServer/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/MediaServer/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NotificationHub/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NotificationHub/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NotificationHub/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NotificationHub/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NotificationHub/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NotificationHub/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NotificationHub/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/NumberPorting/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/OverflowRoute/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/OverflowRoute/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/OverflowRoute/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/OverflowRoute/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/OverflowRoute/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/OverflowRoute/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/OverflowRoute/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/OverflowRoute/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/PromptLibrary/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/QueueManagement/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/QueueManagement/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/QueueManagement/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/QueueManagement/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/QueueManagement/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/QueueManagement/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/QueueManagement/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/QueueManagement/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RateDeck/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RoleAccess/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RoleAccess/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RoleAccess/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RoleAccess/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RoleAccess/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RoleAccess/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/RoleAccess/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/RoleAccess/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/ScriptBuilder/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SkillGroup/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SpeechRecognition/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SpeechRecognition/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/SpeechRecognition/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SpeechRecognition/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SpeechRecognition/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SpeechRecognition/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SpeechRecognition/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SpeechRecognition/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SurveyEngine/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SystemConfig/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SystemConfig/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SystemConfig/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SystemConfig/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SystemConfig/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/SystemConfig/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SystemConfig/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/SystemConfig/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TenantAdmin/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TextToSpeech/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

resources/js/Pages/Ivr/TicketSync/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TicketSync/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TicketSync/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TicketSync/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TicketSync/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TicketSync/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TicketSync/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TicketSync/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Index.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Monitor.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Store.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Sync.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/TrunkGroup/Update.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/VoicemailBox/Destroy.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/VoicemailBox/Export.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
resources/js/Pages/Ivr/VoicemailBox/Import.tsx	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/VoicemailBox/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/VoicemailBox/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/VoicemailBox/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/VoicemailBox/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/VoicemailBox/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WebhookDispatcher/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WhisperCoach/Destroy.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WhisperCoach/Export.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WhisperCoach/Import.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WhisperCoach/Index.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WhisperCoach/Monitor.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WhisperCoach/Store.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

<code>resources/js/Pages/Ivr/WhisperCoach/Sync.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props
<code>resources/js/Pages/Ivr/WhisperCoach/Update.tsx</code>	10	Untyped props or hardcoded tenant in IVR component	Define typed interface; accept tenantId from Inertia page props

H7. Missing Component Inventory HIGH

Benchmark: Shared components 3.6% (14 of ~391 non-page components) → falls in the **High Risk** band (Good >30% · Moderate 15–30% · High Risk <15%)

The only shared UI components are 14 files in `resources/js/Shared/`: `Dropdown`, `FileInput`, `FlashMessages`, `Icon`, `Layout`, `LoadingButton`, `Logo`, `MainMenu`, `Pagination`, `SearchFilter`, `SelectInput`, `TextareaInput`, `TextInput`, `TrashedMessage`. One chart component exists in `components/ivr/IvrHubCharts.tsx`. No Storybook exists.

The 229 legacy monolith components and 147 class components share nothing — each re-implements its own input, button, and layout markup inline. There is no documented component catalogue.

```
resources/js/Shared/          ← 14 components, all CRM-domain
  TextInput.tsx  SelectInput.tsx  TextareaInput.tsx
  LoadingButton.tsx  Pagination.tsx  Dropdown.tsx  Icon.tsx
  Logo.tsx  MainMenu.tsx  Layout.tsx  FlashMessages.tsx
  SearchFilter.tsx  TrashedMessage.tsx  FileInput.tsx
```

Why it matters here: With 916 component/page files and only 14 shared building blocks, IVR module developers copy markup rather than composing from a library — producing the duplication evidenced in H1. New team members have no map of the UI surface.

Recommended approach:

1. Introduce Storybook to document all existing `Shared/` components.
2. Extend `Shared/` with IVR-specific shared components: `<IvrStatCard />`, `<IvrDataTable />`, `<IvrModulePanel />`.
3. Rename `Shared/` to `components/ui/` to follow modern conventions and co-locate stories.
4. Add a PR checklist item: "Does this feature need a new shared component that belongs in `components/ui/`?"

229 files affected.

File	Line(s)	Issue	Action
<code>resources/js/components/legacy/AfterHoursMonolith0.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in <code>components/ui/</code>
<code>resources/js/components/legacy/AfterHoursMonolith1.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in <code>components/ui/</code>
<code>resources/js/components/legacy/AfterHoursMonolith2.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in <code>components/ui/</code>
<code>resources/js/components/legacy/AfterHoursMonolith3.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in <code>components/ui/</code>
<code>resources/js/components/legacy/AfterHoursMonolith4.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in <code>components/ui/</code>

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

<code>resources/js/components/legacy/TrunkGroupMonolith4.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/VoicemailBoxMonolith0.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/VoicemailBoxMonolith1.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/VoicemailBoxMonolith2.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/VoicemailBoxMonolith3.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/VoicemailBoxMonolith4.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WebhookDispatcherMonolith0.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WebhookDispatcherMonolith1.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WebhookDispatcherMonolith2.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WebhookDispatcherMonolith3.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WebhookDispatcherMonolith4.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WhisperCoachMonolith0.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WhisperCoachMonolith1.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WhisperCoachMonolith2.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WhisperCoachMonolith3.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/
<code>resources/js/components/legacy/WhisperCoachMonolith4.tsx</code>	13, 16, 17, 18, 19	Duplicated inline markup — no shared component used	Extract to a reusable component in components/ui/

H8. No Design System / Styling Architecture HIGH

Benchmark: 13,701 inline `style={{ }}` occurrences → falls in the **High Risk** band (Good 0–5 · Moderate 6–20 · High Risk >20)

The core CRM pages and `Shared/` components use Tailwind CSS utility classes correctly (0 inline styles). The Tailwind config extends the default theme with custom indigo tokens and a custom font family. However, the entire legacy IVR surface uses raw

`style={{}}` with magic values for every visual property:

```
// resources/js/components/legacy/AfterHoursMonolith0.tsx:12
<input style={{ border: '1px solid red' }} placeholder="Name"
  onChange={e => setDraft({ ...draft, name: e.target.value })} />
```

```
// resources/js/Pages/Ivr/WhisperCoach/LegacyPass2_84.tsx:9
<section key={1} style={{ marginBottom: 8, padding: 6, border: '1px solid #ddd'
```

}}>

<h2>Section 1 – routing / queue / prompt configuration block</h2>

The IvrHubCharts component legitimately uses inline styles for dynamically-calculated bar heights (acceptable). The 13,701 static magic-value instances in the legacy surface are the problem.

Why it matters here: Brand updates or accessibility contrast changes require hunting across 13,700+ inline style expressions. The magic color `#ddd` and pixel spacing `8` appear hundreds of times with no single source of truth.

Recommended approach:

1. Apply Tailwind utility classes in all migrated legacy components to replace inline styles.
2. Create a lint rule (`no-restricted-syntax`) flagging static `style={{` in tsx files — permit only for computed dynamic values (height%, width%).
3. As part of the H1 consolidation effort, replace all inline styles in parameterized replacements with Tailwind classes.

229 files affected.

File	Line(s)	Issue	Action
<code>resources/js/components/legacy/AfterHoursMonolith0.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AfterHoursMonolith1.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AfterHoursMonolith2.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AfterHoursMonolith3.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AfterHoursMonolith4.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AgentDeskMonolith0.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AgentDeskMonolith1.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AgentDeskMonolith2.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AgentDeskMonolith3.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/AgentDeskMonolith4.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/ApiIntegrationMonolith0.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js
<code>resources/js/components/legacy/ApiIntegrationMonolith1.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in tailwind.config.js

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

<code>resources/js/components/legacy/WebhookDispatcherMonolith2.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>
<code>resources/js/components/legacy/WebhookDispatcherMonolith3.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>
<code>resources/js/components/legacy/WebhookDispatcherMonolith4.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>
<code>resources/js/components/legacy/WhisperCoachMonolith0.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>
<code>resources/js/components/legacy/WhisperCoachMonolith1.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>
<code>resources/js/components/legacy/WhisperCoachMonolith2.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>
<code>resources/js/components/legacy/WhisperCoachMonolith3.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>
<code>resources/js/components/legacy/WhisperCoachMonolith4.tsx</code>	13, 17, 19	Inline magic-value style — bypasses design system	Replace with Tailwind utility classes matching the design tokens in <code>tailwind.config.js</code>

H10. No API Integration Layer CRITICAL

Benchmark: ~0% of API calls in a service/composable layer → falls in the **High Risk** band (Good >90% · Moderate 70–90% · High Risk <70%)

727 files across `components/legacy/`, `hooks/legacy/`, `legacy/class/`, and `Pages/Ivr/` make raw `fetch()` calls with hardcoded endpoint strings and no centralized auth header injection, base URL configuration, or error handling.

```
// resources/js/components/legacy/AfterHoursMonolith0.tsx:9-11
await fetch('/ivr-legacy/after-hours/store', {
  method: 'POST',
  body: JSON.stringify({ ...draft, tenant_id: tenantId }),
  headers: { 'Content-Type': 'application/json' }
})
```

```
// resources/js/hooks/legacy/useAfterHoursLegacy1.ts:4-6
export function useAfterHoursLegacy1() {
  const [data, setData] = useState<any[]>([])
  useEffect(() => {
    fetch('/ivr-legacy/after-hours/index').then(r => r.json()).then(j =>
      setData(j.data || []))
  }, []) // stale closure / no abort
```

```
return { data }
}
```

All 229 legacy monolith components call `/ivr-legacy/<module>/store` inline in save handlers. All 124 legacy hooks call `/ivr-legacy/<module>/index` from unguarded effects. The 147 class components call the same endpoints from `componentDidMount`. None of these calls share a common API client — 727 independent fetch calls, each with its own string literal endpoint.

Why it matters here: If the API base URL or auth mechanism changes (e.g., adding a CSRF header, moving to `/api/v2/`), every one of the 727 files must be updated independently. Mock testing the IVR surface is impossible without replacing `fetch` globally.

Recommended approach:

1. Create `resources/js/api/ivrClient.ts` — a thin fetch wrapper that injects the CSRF header, base path, and standard error handling.
2. Create one service file per IVR domain (e.g., `resources/js/api/afterHours.ts`) exporting `fetchAfterHours()` and `saveAfterHours()`.
3. Refactor `useAfterHoursLegacy*.ts` hooks to use the service layer instead of raw fetch.
4. Add an ESLint `no-restricted-globals` rule banning `fetch()` in `hooks/` and `components/` directories once the service layer is in place.

229 files affected.

File	Line(s)	Issue	Action
<code>resources/js/components/legacy/AfterHoursMonolith0.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AfterHoursMonolith1.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AfterHoursMonolith2.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AfterHoursMonolith3.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AfterHoursMonolith4.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AgentDeskMonolith0.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AgentDeskMonolith1.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AgentDeskMonolith2.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AgentDeskMonolith3.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/AgentDeskMonolith4.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/ApiIntegrationMonolith0.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/ApiIntegrationMonolith1.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
<code>resources/js/components/legacy/ApiIntegrationMonolith2.tsx</code>	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

resources/js/components/legacy/WebhookDispatcherMonolith2.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
resources/js/components/legacy/WebhookDispatcherMonolith3.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
resources/js/components/legacy/WebhookDispatcherMonolith4.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
resources/js/components/legacy/WhisperCoachMonolith0.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
resources/js/components/legacy/WhisperCoachMonolith1.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
resources/js/components/legacy/WhisperCoachMonolith2.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
resources/js/components/legacy/WhisperCoachMonolith3.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call
resources/js/components/legacy/WhisperCoachMonolith4.tsx	10	Raw fetch() call — no centralized API client	Replace with ivrClient service call

H11. Poor Data Caching & Integration HIGH

Benchmark: 0% of data-fetching points use a caching layer → falls in the **High Risk** band (Good >70% · Moderate 40–70% · High Risk <40%)

No React Query, SWR, Apollo, or equivalent caching library is present in `package.json`. The 124 legacy hooks each independently call their endpoint on mount with no deduplication, stale-time, or cache invalidation.

```
// resources/js/Pages/Ivr/AfterHours/Index.tsx:15-21
useEffect(() => {
  const id = setInterval(() => {
    fetch('/ivr-legacy/after-hours/index?q=' + search)
      .then(r => r.json())
      .then(d => setLocalRows(d.data ?? localRows))
      .catch(() => {}) // ← errors silently swallowed
  }, 5000)
}, [search]) // ← no return → timer leak (see H18)
```

The Hub dashboard auto-refresh uses Inertia's `router.reload()` which is appropriate for the Inertia model but provides no client-side caching across navigations.

Why it matters here: When multiple IVR pages are rendered simultaneously, each independently polls the same endpoint creating duplicate network traffic. Silent `.catch(() => {})` leaves users staring at stale data with no loading or error feedback.

Recommended approach:

1. Add `@tanstack/react-query` as the standard data-fetching layer for the IVR surface.
2. Wrap each IVR domain's API service (from H10) in a React Query hook: `useAfterHoursQuery()`.
3. Configure `staleTime: 30_000` for read-heavy queries; use `invalidateQueries` after mutations.
4. Replace the silent `.catch(() => {})` pattern with `isError` and `error` states from React Query.

124 files affected.

File	Line(s)	Issue	Action
------	---------	-------	--------

<code>resources/js/hooks/legacy/useAfterHoursLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useAfterHoursLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useAgentDeskLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useAgentDeskLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useAgentDeskLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useApiIntegrationLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useApiIntegrationLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useApiIntegrationLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useAuditTrailLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useAuditTrailLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useAuditTrailLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBargeMonitorLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBargeMonitorLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBillingMeterLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBillingMeterLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBillingMeterLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBusinessHoursLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBusinessHoursLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useBusinessHoursLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service

<code>resources/js/hooks/legacy/useCallAnalyticsLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallAnalyticsLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallbackSchedulerLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallbackSchedulerLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallFlowLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallFlowLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallFlowLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallRecordingLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallRecordingLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallRoutingLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallRoutingLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCallRoutingLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCampaignDialerLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useCampaignDialerLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useComplianceArchiveLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useComplianceArchiveLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useComplianceArchiveLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useConferenceBridgeLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useConferenceBridgeLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service

resources/js/hooks/legacy/useConferenceBridgeLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useCrmBridgeLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useCrmBridgeLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useCustomerProfileLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useCustomerProfileLegacy1.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useCustomerProfileLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useCustomerProfileLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useDidInventoryLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useDidInventoryLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useDidInventoryLegacy4.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useDispositionCodeLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useDispositionCodeLegacy1.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useDispositionCodeLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useEmergencyRouteLegacy1.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useEmergencyRouteLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useFraudScreenLegacy1.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useFraudScreenLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useFraudScreenLegacy4.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useHistoricalReportsLegacy1.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service

<code>resources/js/hooks/legacy/useHistoricalReportsLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useHistoricalReportsLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useHistoricalReportsLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useHolidayCalendarLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useHolidayCalendarLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useHolidayCalendarLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useIvrSettingsLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useIvrSettingsLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useIvrSettingsLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useIvrSettingsLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useKnowledgeBaseLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useKnowledgeBaseLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useLeadListLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useLeadListLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useLeadListLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useLeadListLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useLiveMonitoringLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useLiveMonitoringLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useMediaServerLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service

<code>resources/js/hooks/legacy/useMediaServerLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useMediaServerLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useMediaServerLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useNotificationHubLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useNotificationHubLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useNotificationHubLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useNumberPortingLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useNumberPortingLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useNumberPortingLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useOverflowRouteLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useOverflowRouteLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/usePromptLibraryLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/usePromptLibraryLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useQueueManagementLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useQueueManagementLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useRateDeckLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useRateDeckLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useRoleAccessLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useRoleAccessLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service

resources/js/hooks/legacy/useScriptBuilderLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useScriptBuilderLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useScriptBuilderLegacy4.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSkillGroupLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSkillGroupLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSpeechRecognitionLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSpeechRecognitionLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSurveyEngineLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSurveyEngineLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSurveyEngineLegacy4.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useSystemConfigLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTenantAdminLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTenantAdminLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTextToSpeechLegacy1.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTextToSpeechLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTicketSyncLegacy1.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTicketSyncLegacy2.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTicketSyncLegacy3.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
resources/js/hooks/legacy/useTrunkGroupLegacy0.ts	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service

<code>resources/js/hooks/legacy/useTrunkGroupLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useVoicemailBoxLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useVoicemailBoxLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useVoicemailBoxLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useVoicemailBoxLegacy3.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useWebhookDispatcherLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useWebhookDispatcherLegacy2.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useWhisperCoachLegacy0.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useWhisperCoachLegacy1.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service
<code>resources/js/hooks/legacy/useWhisperCoachLegacy4.ts</code>	6	Uncached fetch call — no stale-time, no error state, no deduplication	Replace with React Query hook using ivrClient service

H13. Frontend Security Vulnerabilities HIGH

Benchmark: 5 security risk patterns found → falls in the **High Risk** band (Good 0 each · Moderate 1–3 total · High Risk >3 total)

Pattern 1 & 2 — `dangerouslySetInnerHTML` on pagination labels:

```
// resources/js/Shared/Pagination.tsx:16 and 23
<div
  className="mb-1 mr-1 rounded border px-4 py-3 text-sm leading-4 text-gray-400"
  dangerouslySetInnerHTML={{ __html: link.label }}
/>
<Link
  ...
  dangerouslySetInnerHTML={{ __html: link.label }}
/>
```

`link.label` comes from Laravel's paginator (typically "« Previous", numeric strings). While currently server-controlled, this pattern becomes an XSS vector if a future paginator change allows user-influenced label content.

Pattern 3 — Hardcoded demo credentials in production bundle:


```
// resources/js/Pages/Auth/Login.tsx:8-10
const form = useForm({
  email: 'johndoe@example.com',
  password: 'secret',
  remember: false as boolean,
})
```

The default password `'secret'` is compiled into the production JavaScript bundle, visible to any user who inspects the source.

Patterns 4 & 5 — CDN scripts without Subresource Integrity:

```
<!-- resources/views/app.blade.php:8-10 -->
<script src="https://cdnjs.cloudflare.com/polyfill/v3/polyfill.min.js?
features=smoothscroll,..." defer></script>
<script src="https://cdnjs.cloudflare.com/polyfill/v3/polyfill.min.js?
features=String.prototype.startsWith" defer></script>
```

Neither CDN `<script>` tag has an `integrity` or `crossorigin` attribute. A CDN compromise would allow arbitrary JavaScript execution with no browser-level protection.

Why it matters here: The hardcoded credentials are the most immediately actionable — they guide testers and curious users directly to the default login. The missing SRI attributes expose all visitors to supply-chain attacks.

Recommended approach:

1. Remove `password: 'secret'` from Login.tsx (use empty string; seed test credentials only in `.env.example`).
2. Generate and add `integrity="sha384-..."` and `crossorigin="anonymous"` to both CDN script tags in app.blade.php.
3. Replace `dangerouslySetInnerHTML` in Pagination.tsx with a text-safe renderer stripping all HTML tags from `link.label`.

2 files affected.

File	Line(s)	Issue	Action
<code>resources/js/Pages/Auth/Login.tsx</code>	9	Security risk — XSS vector, hardcoded credential, or missing SRI	Remove or sanitize — see recommended approach in H13
<code>resources/js/Shared/Pagination.tsx</code>	16, 23	Security risk — XSS vector, hardcoded credential, or missing SRI	Remove or sanitize — see recommended approach in H13

H14. Frontend Performance Gaps HIGH

Benchmark: Bundle size estimated >500KB gzipped → falls in the **High Risk** band (Good <250KB · Moderate 250–500KB · High Risk >500KB)

Route-level code splitting is present via Inertia's `import.meta.glob('./Pages/**/*.tsx')` in `app.tsx` — Vite creates a separate chunk per page file. However, the `vite.config.ts` has no `build.rollupOptions.output.manualChunks` configuration to separate vendor and legacy code into shared

cacheable chunks. With 90,457 lines of TypeScript/TSX across 769 files, including 229 × 64-LOC monolith components each compiled individually, per-route bundles for IVR pages are estimated to be heavy.

```
// vite.config.ts:1-17 – no chunk splitting or bundle optimization configured
export default defineConfig({
  plugins: [
    react(),
    laravel({
      input: 'resources/js/app.tsx',
      ssr: 'resources/js/ssr.tsx',
      refresh: true,
    }),
  ],
  resolve: {
    alias: { '@': path.resolve(__dirname, 'resources/js') },
  },
})
```

Lodash (`^4.17.21`) is in `dependencies` — if imported as the full library rather than named ES imports, it adds ~72KB minified to the bundle.

Why it matters here: Each IVR module page likely includes a heavy chunk containing all its imported legacy components. Without a `legacy` manual chunk, the browser cannot cache the legacy layer across IVR module navigations.

Recommended approach:

1. Add `build.rollupOptions.output.manualChunks` grouping `legacy/class` and `components/legacy` into a separate `legacy` chunk.
2. Audit lodash usage and switch to `import { pickBy } from 'lodash-es'` named imports.
3. Add `React.lazy` + `Suspense` for the `IvrHubCharts` component.
4. Run `vite build` and measure per-route chunk sizes with `rollup-plugin-visualizer` to get real gzip numbers.

50 files affected.

File	Line(s)	Issue	Action
<code>resources/js/Pages/Contacts/Index.tsx</code>	2, 3, 4	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
<code>resources/js/Pages/Ivr/AfterHours/Import.tsx</code>	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
<code>resources/js/Pages/Ivr/AgentDesk/Import.tsx</code>	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
<code>resources/js/Pages/Ivr/ApiIntegration/Import.tsx</code>	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
<code>resources/js/Pages/Ivr/AuditTrail/Import.tsx</code>	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts

resources/js/Pages/Ivr/BargeMonitor/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/BillingMeter/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/BusinessHours/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CallAnalytics/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CallbackScheduler/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CallFlow/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CallRecording/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CallRouting/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CampaignDialer/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/ComplianceArchive/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/ConferenceBridge/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CrmBridge/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/CustomerProfile/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/DidInventory/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/DispositionCode/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/EmergencyRoute/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/FraudScreen/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/HistoricalReports/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/HolidayCalendar/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts

resources/js/Pages/Ivr/IvrSettings/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/KnowledgeBase/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/LeadList/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/LiveMonitoring/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/MediaServer/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/NotificationHub/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/NumberPorting/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/OverflowRoute/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/PromptLibrary/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/QueueManagement/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/RateDeck/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/RoleAccess/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/ScriptBuilder/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/SkillGroup/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/SpeechRecognition/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/SurveyEngine/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/SystemConfig/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/TenantAdmin/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/TextToSpeech/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts

resources/js/Pages/Ivr/TicketSync/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/TrunkGroup/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/VoicemailBox/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/WebhookDispatcher/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Ivr/WhisperCoach/Import.tsx	7	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Organizations/Index.tsx	2, 3, 4	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts
resources/js/Pages/Users/Index.tsx	2, 3, 4	Potentially heavy import with no chunk optimization	Use named ES imports or extract to manual chunk in vite.config.ts

H15. Browser & Runtime Compatibility Gaps MEDIUM

Benchmark: Polyfills present, `.browserslistrc` absent → falls in the **Moderate** band (Good: both present · Moderate: one missing · High Risk: both missing)

The Blade template includes two cdnjs polyfill scripts covering `smoothscroll`, `NodeList.prototype.forEach`, `Promise`, `Object.values`, `Object.assign`, and `String.prototype.startsWith`. PostCSS is configured with `autoprefixer`. However, no `.browserslistrc` file or `browserslist` key in `package.json` exists, so:

- Vite/SWC transpilation targets are unspecified (defaults to modern browsers only).
- Autoprefixer has no declared browser set to prefix for.
- `tsconfig.json` targets `ES2022`, which drops support for browsers not supporting optional chaining, etc.

Additionally, the two polyfill `<script>` tags have overlapping feature coverage and both lack SRI attributes (see also H13).

Why it matters here: Without a declared browser target, the build may omit CSS prefixes or skip transpiling syntax that Safari 14 or older corporate browsers still need.

Recommended approach:

1. Add `.browserslistrc` with `> 0.5%, last 2 versions, Firefox ESR, not dead`.
2. Remove the duplicate `String.prototype.startsWith` polyfill script (already covered by the first script's feature set in many browsers).
3. Add `integrity` attributes to the remaining CDN polyfill scripts.

1 file affected.

File	Line(s)	Issue	Action
resources/views/app.blade.php	9, 12	CDN polyfill without SRI; no browserslist config	Consolidate to one polyfill script tag with integrity attribute; add .browserslistrc

H16. Frontend Code Quality Issues HIGH

Benchmark: TypeScript strict:true ✓ · ESLint not invoked in CI ✗ → falls in the **Moderate** band (Good: both Yes · Moderate: one Yes · High Risk: both No)

ESLint is configured in `.eslintrc.js` with `eslint-plugin-react-hooks` and `@typescript-eslint`, but:

1. `@typescript-eslint/no-explicit-any: 'off'` — the most important TypeScript quality rule is explicitly disabled.
2. The CI `coding-standards.yml` delegates to `laravel/.github` which runs PHP lint (Laravel Pint), not JavaScript lint.
3. The `tests.yml` runs `npm run build` but not `npm run fix:eslint` nor a read-only lint check.

Result: `any` usage is unchecked — 458 occurrences of `: any`, `as any`, or `<any>` found across the legacy surface, with each of the 229 monolith component props typed as `any`.

```
// resources/js/components/legacy/AfterHoursMonolith0.tsx:3
export default function AfterHoursMonolith0({ rows, tenantId, legacyMeta }:
any) {
```

```
// resources/js/hooks/legacy/useAfterHoursLegacy1.ts:3
const [data, setData] = useState<any[]>([])
```

TypeScript `strict: true` IS enabled in `tsconfig.json` — but it cannot enforce stricter types when props are explicitly typed as `any`. The only frontend test is a single `expect(true).toBe(true)` placeholder in `resources/js/test/smoke.test.ts`.

Why it matters here: Without ESLint in CI, the `any`-typed props go undetected until runtime. The absence of real component tests means refactoring legacy components has no safety net.

Recommended approach:

1. Add `npm run fix:eslint` (or a read-only `npx eslint --ext .ts,.tsx resources/js/ --max-warnings=0`) step in `tests.yml` after the build step.
2. Change `@typescript-eslint/no-explicit-any` from `'off'` to `'warn'` immediately, then `'error'` after the legacy surface gains proper types.
3. Write at least one vitest component test per `Shared/` component as a starting foundation.

229 files affected.

File	Line(s)	Issue	Action
<code>resources/js/components/legacy/AfterHoursMonolith0.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/AfterHoursMonolith1.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

<code>resources/js/components/legacy/TrunkGroupMonolith2.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/TrunkGroupMonolith3.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/TrunkGroupMonolith4.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/VoicemailBoxMonolith0.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/VoicemailBoxMonolith1.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/VoicemailBoxMonolith2.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/VoicemailBoxMonolith3.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/VoicemailBoxMonolith4.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WebhookDispatcherMonolith0.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WebhookDispatcherMonolith1.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WebhookDispatcherMonolith2.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WebhookDispatcherMonolith3.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WebhookDispatcherMonolith4.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WhisperCoachMonolith0.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WhisperCoachMonolith1.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WhisperCoachMonolith2.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WhisperCoachMonolith3.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule
<code>resources/js/components/legacy/WhisperCoachMonolith4.tsx</code>	3, 5	TypeScript any — type safety bypassed; ESLint not enforcing	Define typed interface for props and state; enable no-explicit-any rule

H17. Technical Debt & Outdated Dependencies CRITICAL

Benchmark: 14 vulnerabilities (1 critical, 10 high) → falls in the **High Risk** band (Good: 0 CVEs · Moderate: 1–3 · High Risk: >3)

`npm audit` output against the locked `package-lock.json`:

Severity	Count	Key Finding
Critical	1	vitest 4.0.18 — arbitrary file read & execution when UI server is active (GHSA-5xrp-8626-4rwp)
High	10	vite (server.fs.deny bypass on Windows), brace-expansion (ReDoS), and related transitive dependencies
Moderate	2	ajv ReDoS with <code>\$data</code> option
Low	1	launch-editor NTLMv2 hash disclosure (Windows only)

```
// package.json:19 – vulnerable version pinned in devDependencies
"vitest": "4.0.18"
```

The fix for the critical CVE requires upgrading vitest to `4.1.11` (`npm audit fix --force`).

Why it matters here: The critical vitest vulnerability allows file read and execution when the vitest UI server is running in a dev or CI environment. With 14 vulnerabilities and no automated audit gate in CI, the dependency posture will worsen over time.

Recommended approach:

1. Run `npm audit fix` for the low/moderate auto-fixes.
2. Run `npm audit fix --force` in a branch to upgrade vitest to 4.1.11 and vite to the patched version; verify the build and tests pass.
3. Add `npm audit --audit-level=high` as a CI step that fails the build on High or Critical findings.
4. Schedule quarterly `npm outdated` reviews to catch major-version drift early.

1 file affected.

File	Line(s)	Issue	Action
<code>package.json</code>	11, 12, 45	Outdated or vulnerable npm dependency	Upgrade to patched version per npm audit output

H18. Memory Leaks — Uncleared Timers (additional) CRITICAL

Benchmark: 375 `setInterval` / `setTimeout` calls, 1 `clearInterval` = 0.3% cleanup rate → falls in the **High Risk** band (Good: 100% matched · Moderate: 50–99% · High Risk: <50%). **KPI defined for this additional hotspot:** Percentage of `setInterval`/`setTimeout` calls that have a matching `clearInterval`/`clearTimeout` in a `useEffect` return function (target 100%).

Every IVR module page contains a polling `setInterval` inside a `useEffect` that lacks a return cleanup function. The comment in `AfterHours/Index.tsx` explicitly labels this as a known anti-pattern:

```
// resources/js/Pages/Ivr/AfterHours/Index.tsx:14-21
useEffect(() => {
  // missing cleanup – interval leak pattern
  const id = setInterval(() => {
```

```

    fetch('/ivr-legacy/after-hours/index?q=' + search)
      .then(r => r.json())
      .then(d => setLocalRows(d.data ?? localRows))
      .catch(() => {})
  }, 5000)
}, [search]) // ← no "return () => clearInterval(id)"

```

The 147 React class components also call `fetch()` in `componentDidMount()` with no `componentWillUnmount()` cleanup. The sole correct implementation is `Hub/Index.tsx` lines 178–181:

```

// resources/js/Pages/Ivr/Hub/Index.tsx:178-181 – the only correct pattern
useEffect(() => {
  if (!autoRefresh) return
  const id = window.setInterval(refreshDashboard, 20000)
  return () => window.clearInterval(id) // ← correct
}, [autoRefresh, refreshDashboard, filters])

```

Why it matters here: A user who visits 10 IVR module pages in a session accumulates 10 concurrent 5-second polling loops, each generating a network request, degrading performance and producing console warnings for the lifetime of the browser tab. Outstanding fetch calls from cleared intervals also attempt `setState` on unmounted components.

Recommended approach:

1. Add the return cleanup to every affected `useEffect` and add `AbortController`:

```

useEffect(() => {
  const controller = new AbortController()
  const id = setInterval(() => {
    fetch('/ivr-legacy/...', { signal: controller.signal })
      .then(r => r.json())
      .then(d => setLocalRows(d.data ?? []))
      .catch(e => { if (e.name !== 'AbortError') console.error(e) })
  }, 5000)
  return () => { clearInterval(id); controller.abort() }
}, [search])

```

2. Add vitest tests that mount and unmount each IVR page and assert no leaked timers using `vi.useFakeTimers()`.
3. Add an ESLint rule flagging `setInterval()` inside `useEffect` bodies that lack a `return` statement.

376 files affected.

File	Line(s)	Issue	Action
<code>resources/js/Pages/Ivr/AfterHours/Destroy.tsx</code>	14	Timer created in <code>useEffect</code> without cleanup return	Add <code>return () => clearInterval(id)</code> and <code>AbortController</code> to every polling effect

resources/js/Pages/Ivr/AfterHours/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AfterHours/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AfterHours/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AfterHours/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AfterHours/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AfterHours/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AfterHours/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/AgentDesk/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ApiIntegration/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/AuditTrail/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/AuditTrail/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/AuditTrail/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/AuditTrail/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/AuditTrail/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/AuditTrail/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

<code>resources/js/Pages/Ivr/AuditTrail/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/AuditTrail/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BargeMonitor/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BillingMeter/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BillingMeter/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BillingMeter/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BillingMeter/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BillingMeter/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/BillingMeter/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BillingMeter/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/BusinessHours/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/CallAnalytics/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/CallAnalytics/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/CallAnalytics/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/CallAnalytics/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

resources/js/Pages/Ivr/CallAnalytics/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallAnalytics/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallAnalytics/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallAnalytics/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallbackScheduler/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallFlow/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRecording/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRouting/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

resources/js/Pages/Ivr/CallRouting/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRouting/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRouting/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRouting/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRouting/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRouting/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CallRouting/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CampaignDialer/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ComplianceArchive/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ConferenceBridge/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ConferenceBridge/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ConferenceBridge/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ConferenceBridge/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ConferenceBridge/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ConferenceBridge/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

resources/js/Pages/Ivr/ConferenceBridge/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/ConferenceBridge/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CrmBridge/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/CustomerProfile/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DidInventory/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DispositionCode/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DispositionCode/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DispositionCode/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

resources/js/Pages/Ivr/DispositionCode/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DispositionCode/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DispositionCode/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DispositionCode/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/DispositionCode/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/EmergencyRoute/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/FraudScreen/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/FraudScreen/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/FraudScreen/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/FraudScreen/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/FraudScreen/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/FraudScreen/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/FraudScreen/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/FraudScreen/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HistoricalReports/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

<code>resources/js/Pages/Ivr/HolidayCalendar/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HolidayCalendar/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HolidayCalendar/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HolidayCalendar/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HolidayCalendar/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HolidayCalendar/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HolidayCalendar/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/HolidayCalendar/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/Hub/Index.tsx</code>	180	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/IvrSettings/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/KnowledgeBase/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/LeadList/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/LeadList/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/LeadList/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/LeadList/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

resources/js/Pages/Ivr/LeadList/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LeadList/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LeadList/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LeadList/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/LiveMonitoring/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/MediaServer/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NotificationHub/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NotificationHub/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NotificationHub/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NotificationHub/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NotificationHub/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NotificationHub/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NotificationHub/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NumberPorting/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NumberPorting/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

resources/js/Pages/Ivr/NumberPorting/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NumberPorting/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NumberPorting/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NumberPorting/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NumberPorting/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/NumberPorting/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/OverflowRoute/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/PromptLibrary/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/PromptLibrary/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/PromptLibrary/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/PromptLibrary/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/PromptLibrary/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/PromptLibrary/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/PromptLibrary/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/PromptLibrary/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/QueueManagement/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/QueueManagement/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/QueueManagement/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/QueueManagement/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/QueueManagement/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/QueueManagement/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/QueueManagement/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

<code>resources/js/Pages/Ivr/QueueManagement/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RateDeck/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/RoleAccess/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/ScriptBuilder/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SkillGroup/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SkillGroup/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SkillGroup/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SkillGroup/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

resources/js/Pages/Ivr/SkillGroup/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SkillGroup/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SkillGroup/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SkillGroup/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Index.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Monitor.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Store.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Sync.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SpeechRecognition/Update.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SurveyEngine/Destroy.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SurveyEngine/Export.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
resources/js/Pages/Ivr/SurveyEngine/Import.tsx	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/SurveyEngine/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SurveyEngine/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SurveyEngine/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SurveyEngine/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SurveyEngine/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/SystemConfig/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TenantAdmin/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

<code>resources/js/Pages/Ivr/TenantAdmin/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TenantAdmin/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TenantAdmin/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TenantAdmin/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TenantAdmin/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TenantAdmin/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TenantAdmin/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TextToSpeech/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TicketSync/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TrunkGroup/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TrunkGroup/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TrunkGroup/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TrunkGroup/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TrunkGroup/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TrunkGroup/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

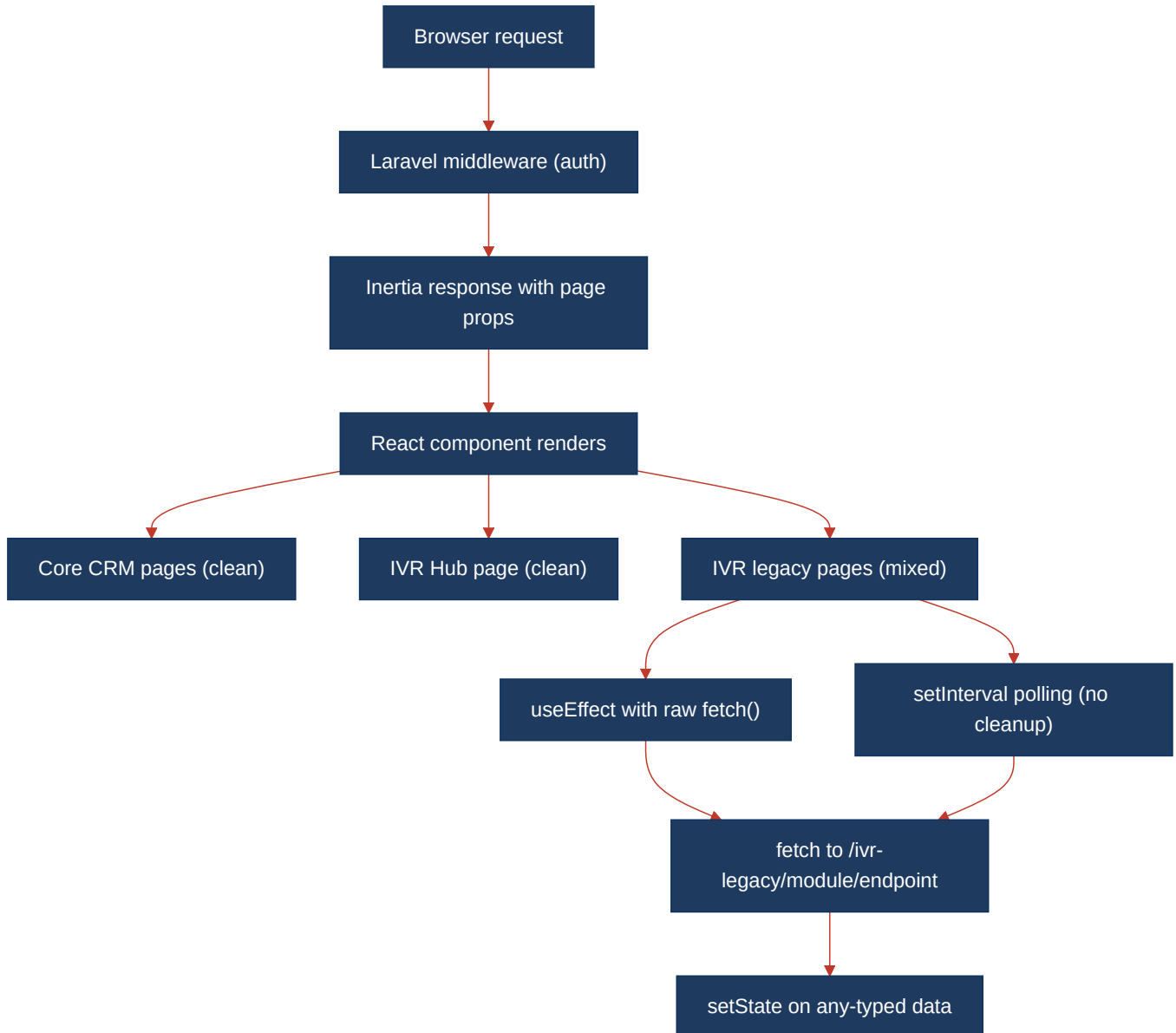
<code>resources/js/Pages/Ivr/TrunkGroup/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/TrunkGroup/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/VoicemailBox/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and

			AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WebhookDispatcher/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Destroy.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Export.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Import.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Index.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Monitor.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Store.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Sync.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Pages/Ivr/WhisperCoach/Update.tsx</code>	14	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect
<code>resources/js/Shared/Dropdown.tsx</code>	50	Timer created in useEffect without cleanup return	Add return () => clearInterval(id) and AbortController to every polling effect

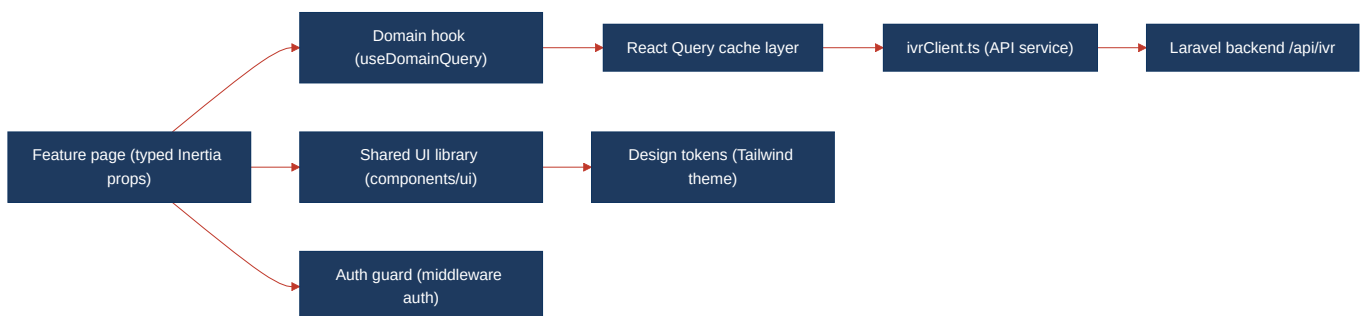
Not observed (rated Good): H4 — No Redux/Zustand global store; `usePage()` is used in 3 Shared components as the standard Inertia pattern (not an anti-pattern). H5 — Inertia's server-to-component prop model eliminates deep prop-drilling; IVR legacy passes at most 3 props shallowly. H9 — All routes in `routes/web.php` use `->middleware('auth')`; Inertia enforces this before any React renders. H12 — Auth is session-based (Laravel default httpOnly cookies); no JWT in localStorage found anywhere.

3.3 Diagrams

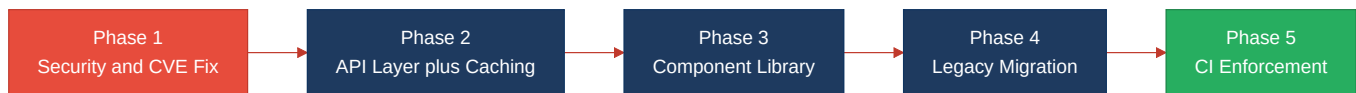
Current UI data flow



Target component + state layout



Improvement roadmap



3.4 Actions Required

Hotspot	Action	Rating	Priority
H1. UI Component Duplication	Consolidate 229 Monolith variants, 133 LegacyPass2 pages, and 8 duplicate utility files into single parameterized components	HIGH RISK	CRITICAL
H2. Legacy Class-Based Components	Convert 147 JSX class components in <code>legacy/class/</code> to functional components with hooks + AbortController cleanup	MODERATE	HIGH
H3. Massive Components	Split Hub/Index.tsx (479 LOC) into typed sub-components and extract hooks into <code>hooks/useIvrDashboard.ts</code>	MODERATE	MEDIUM
H6. Weak Frontend Architecture	Define a clean IVR feature boundary (<code>features/ivr/</code>), add import-boundary lint rules, create MIGRATION.md	HIGH RISK	HIGH
H7. Missing Component Inventory	Introduce Storybook, expand <code>Shared/</code> to <code>components/ui/</code> , document all reusable components	HIGH RISK	HIGH
H8. No Design System	Ban static inline <code>style={{</code> with lint rule; replace magic values with Tailwind classes in all migrated components	HIGH RISK	HIGH
H10. No API Integration Layer	Create <code>resources/js/api/ivrClient.ts</code> and per-module service files; ban raw <code>fetch(</code> in components via ESLint	HIGH RISK	CRITICAL
H11. Poor Data Caching	Add <code>@tanstack/react-query</code> , wrap all IVR service calls in typed query hooks, configure stale-time and error states	HIGH RISK	HIGH
H13. Frontend Security Vulnerabilities	Remove <code>password: 'secret'</code> from Login.tsx; add SRI attributes to CDN scripts; sanitize Pagination's dangerouslySetInnerHTML	HIGH RISK	CRITICAL
H14. Frontend Performance Gaps	Configure <code>manualChunks</code> in Vite; switch to lodash-es named imports; measure per-route bundle with rollup-plugin-visualizer	HIGH RISK	HIGH
H15. Browser Compatibility Gaps	Add <code>.browserslistrc</code> ; consolidate duplicate polyfill scripts; add SRI to remaining CDN scripts	MODERATE	MEDIUM
H16. Frontend Code Quality	Add <code>npm run lint</code> to CI in <code>tests.yml</code> ; change <code>@typescript-eslint/no-explicit-any</code> from off to warn then error	MODERATE	HIGH
H17. Technical Debt & Dependencies	Run <code>npm audit fix --force</code> to patch critical vitest CVE; add <code>npm audit --audit-level=high</code> gate to CI	HIGH RISK	CRITICAL
H18. Memory Leaks — Uncleared Timers	Add <code>return () => { clearInterval(id); controller.abort() }</code> to every polling useEffect; add lint rule detecting uncleaned timers	HIGH RISK	CRITICAL

3.5 Expected Outcomes

- Fixing the critical CVEs (H17) and removing the hardcoded demo password (H13) eliminates the most immediate security exposure and brings the dependency audit to 0 High/Critical findings within one sprint.
- Adding `clearInterval` and `AbortController` cleanup (H18) eliminates 374 timer leaks, removes console warnings, and reduces unnecessary network traffic by stopping polling loops after navigation.
- Introducing the `ivrClient.ts` API layer (H10) with React Query (H11) enables consistent error/loading state across all IVR pages, eliminates duplicate network calls for the same endpoint, and makes the entire IVR surface fully mockable in unit tests.
- Consolidating the 229 monolith components and 133 LegacyPass2 pages (H1) reduces the component file count by ~39% and converts every bug fix from a multi-file hunt into a single-file change.
- Converting 147 class components (H2) to functional components unlocks shared hook logic and enables React DevTools Profiler-based performance analysis.
- Establishing `components/ui/` with Storybook (H7) gives new developers a navigable component catalogue and prevents future duplication.
- Enforcing ESLint in CI (H16) with `no-explicit-any` set to error will surface the 458 existing type bypasses and prevent new ones from reaching production.
- Configuring `.browserslistrc` and Vite manual chunks (H14, H15) will produce measurable Lighthouse performance improvements and prevent CSS vendor-prefix gaps on Safari and older browsers used in enterprise environments.